AgentU: An Event-Sourced Multi-Agent System for LLM-Assisted Recruitment, with an Empirical Study of Its Failure Modes

Authors: M G K Gowtham (2320030018), V Balaji Naidu (2320030016), D Hemanth Raj (2320030004)

Guide: Dr. N Shirisha, Associate Professor

Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation

Engineering Capstone Project

Abstract

Automated resume screening is increasingly delegated to large language models (LLMs), but deployments are rarely accompanied by measurements of how they fail. We present AgentU, a web application in which three cooperating agents automate a small organisation's hiring workflow. An HR Agent turns a company profile and a rough role description into a public job listing, distributes it and screens every application. A Management Agent moves candidates through an interview pipeline and drafts candidate emails that the recruiter approves before sending. An Analysis Agent reports each role's score distribution with confidence intervals and flags applications whose scores should not be trusted. The system rests on two architectural commitments: *event-sourced orchestration*, in which every agent step is appended to a log that the interface projects, so progress visibility is decoupled from the request/response cycle; and *degradation-first design*, in which every external dependency has a documented fallback.

We evaluate the screening component on the production code path against author-assigned ground truth on 48 job–resume pairs, comparing the LLM scorer with the system's own keyword-overlap fallback and a reject-everyone baseline. The LLM scorer reaches 81.3% accuracy, macro F1 0.740, Cohen's $\kappa = 0.623$ and Spearman $\rho = 0.915$, against $\kappa = 0.235$ for keyword overlap and $\kappa = 0$ for the trivial rule. Every error is an over-promotion: no candidate a human would shortlist was rejected. Four negative results are equally central. The scorer does not see through keyword stuffing; it is stable under repetition (0/8 decision flips) but not under rewording of the same facts; a one-line forged system turn moved an unrelated candidate from 0 to 95 (2 of 10 attacks succeeded); and scanned PDFs parse "successfully" while yielding almost no text (78.6% usable against a 100% parse rate). The system's own silent fallback also corrupted our first measurement run.

A revised rubric prompt that states the decision bands, asks for per-requirement evidence and fences resume text as untrusted input raised held-out accuracy from 87.5% to 91.7% and κ from 0.695 to 0.781, and blocked all ten injection attempts. The remaining failure modes are now detected in the product itself: the Analysis Agent flags fallback-scored, unreadable, injection-bearing and keyword-stuffed applications, and every fallback is reported to the recruiter with its cause.

Keywords: LLM-assisted recruitment, multi-agent systems, resume screening, prompt injection, event sourcing, graceful degradation, evaluation methodology

1. Introduction

1.1 Motivation

Recruiting at small and mid-sized organisations involves substantial repetitive work that is adjacent to, but not the same as, judgement: rewriting a rough role description into a presentable public listing, distributing it, collecting applications in varied document formats, reading each resume against the same requirements, scheduling interviews, writing to candidates, and keeping the hiring manager informed. Applicant tracking systems address storage and workflow, but the reading and drafting remain manual, and the informing is typically a dashboard the recruiter must remember to visit.

Large language models are an obvious fit for the drafting and reading. What is less obvious, and much less frequently reported, is how such a system behaves when the model is wrong, when the model is unavailable, when the input document cannot be read, or when the input is adversarial. A candidate has a direct incentive to manipulate a screener, and unlike most LLM applications, the untrusted input here arrives as a file the candidate authored.

1.2 Problem statement

Existing LLM-based screening systems automate a single hiring stage and are evaluated offline for agreement with human ratings [1], [2], without measuring how they fail. We address two questions:

1. **Architecturally:** how should a set of agents that perform a multi-step, partly slow and partly failure-prone workflow report their progress and survive the failure of their dependencies?
2. **Empirically:** how well does LLM-based resume screening agree with human judgement in this system, and, more importantly, *how does it fail*?

1.3 Contributions

1. **A three-agent recruitment system** (HR, Management and Analysis Agents) covering publishing, distribution, screening, candidate coordination and pipeline analysis in one deployment, with the recruiter as the final decision-maker at every consequential step (§3, §5).
2. **An event-sourced agent architecture** in which progress visibility is decoupled from the request/response cycle. Each step appends an `AgentEvent`; the recruiter's activity feed is a projection of that log, so work deferred past the HTTP response is exactly as visible as work performed before it (§4.2, §4.4).
3. **A degradation-first design** in which every external dependency has an explicit fallback that keeps the core workflow available, and in which each fallback is reported to the recruiter with its cause (§4.3).
4. **An evaluation harness that measures the production code path** rather than a reimplementation, together with a methodological finding: a system that degrades silently is a hazard to its own evaluation. Our first run recorded 42 of 48 observations as model output when the model had stopped responding (§6.5, §7.8).
5. **An empirical characterisation of failure modes** in LLM resume screening, including a prompt-injection vulnerability that changed a hiring decision, sensitivity of scores to resume wording rather than substance, and silent upstream extraction failure (§7).
6. **Mitigations evaluated or deployed:** a rubric prompt evaluated on held-out data (§7.9), and an Analysis Agent that turns each measured failure mode into a per-application integrity flag (§5.5).

1.4 Scope

This is a capstone-scale study. The labelled dataset is 48 synthetic pairs plus 24 held-out pairs, with single-annotator ground truth, one model and one temperature setting. Results indicate direction and support design claims; they are not a benchmark. Limitations are stated in full in §9.

2. Related Work

LLM-based resume screening and multi-agent hiring. Lo et al. [1], the base paper for this project, propose a four-agent framework (resume extractor, evaluator, summariser and score formatter) built on CrewAI and LangChain, in which retrieval-augmented generation (RAG) supplies the evaluator with external criteria such as certifications, university rankings and company-specific hiring criteria. On 105 anonymised online resumes rated by HR professionals, the RAG configuration with DeepSeek-V3 reached a Pearson correlation of 0.84 on the top 10% of candidates and a mean absolute error of 0.90, outperforming single-LLM baselines. Gan et al. [2] use LLM agents to summarise and grade resumes, reporting screening 11 times faster than manual review and an F1 of 87.73% for resume sentence classification after fine-tuning. Filatova et al. [3] describe a modular multi-agent hiring assistant with document and video preprocessing, public-data verification, fit scoring with risk penalties

and human-in-the-loop validation; on 64 applicants for a backend role it needed 1.70 hours per qualified candidate against 3.33 hours for an experienced recruiter. MLAR [4] embeds three LLM layers (job-posting features, resume parsing and semantic matching) in a robotic-process-automation applicant tracker and processes a resume in 5.4 s on a 2,400-resume benchmark. General multi-agent frameworks such as AutoGen [5] structure LLM applications as conversations between customisable agents. Work on LLM-as-a-judge [6] shows that a strong model can reach over 80% agreement with human preferences, the level of agreement between humans, while exhibiting position, verbosity and self-enhancement biases.

AgentU follows the evaluation logic of [1], agreement with human judgement rather than raw accuracy, but departs from it in three ways. Its screening step is deliberately a single model call with a deterministic fallback, so that determinism and injection effects can be attributed to one component; its agentic behaviour lies in workflow orchestration rather than in scoring. It covers the stages before and after screening (publishing, distribution, interviews and candidate email). And where [1]–[4] evaluate accuracy or speed, we measure determinism, presentation sensitivity, adversarial input and extraction failure on the code path that runs in production.

Algorithmic hiring, bias and ethics. The best-known cautionary case is Amazon's internal recruiting tool, abandoned after it was found to disadvantage women [7]. Bogen and Rieke [8] survey where bias enters the hiring funnel, and Raghavan et al. [9] find that vendors of pre-employment assessments disclose little about how their models are validated or de-biased. Köchling and Wehner [10] systematically review discrimination in algorithmic HR decisions, and Hunkenschroer and Luetge [11] map the ethical risks of AI-enabled recruiting, including bias, privacy, transparency and accountability. Fabris et al. [12] survey systems, bias measures, mitigation strategies, datasets and law, concluding that whether algorithmic hiring is less biased than low-tech alternatives remains open. Audits of LLM screeners confirm the concern: Armstrong et al. [13] find stereotype-based race and gender bias in GPT-3.5's resume ratings, and Wilson and Caliskan [14] find that embedding-based screening favoured White-associated names in 85.1% of cases and female-associated names in only 11.1%.

Regulation. New York City Local Law 144 requires independent bias audits of automated employment decision tools and notification of candidates [15]. The EU Artificial Intelligence Act classifies AI systems used in recruitment and worker management as high-risk [16]. Both are directly relevant to a system of this shape (§9).

Prompt injection and defences. Perez and Ribeiro [17] showed that handcrafted inputs can hijack GPT-3's goal. Greshake et al. [18] describe indirect prompt injection, in which the attack arrives inside data the application ingests; this is exactly our threat model, since the resume is candidate-authored. The OWASP Top 10 for LLM Applications lists prompt injection as LLM01 [19]. Defences include benchmark-driven boundary awareness and explicit reminders [20], spotlighting (delimiting, datamarking or encoding untrusted text), which reduced attack success from over 50% to under 2% [21], and structured queries that separate prompt and data channels through fine-tuning [22]. Baxi et al. [23] study injection in résumé screening specifically, finding that injected self-promotion helps most when few candidates use it and candidates are of similar quality.

Agreement metrics. Cohen's κ [24] corrects observed agreement for agreement expected by chance, which §6.4 argues is essential under our class imbalance. The Analysis Agent reports band shares with Wilson score intervals [25], which behave well at the small sample sizes of a single role.

Gap. The reviewed work either scores resumes offline [1]–[4], or studies bias [13], [14] and injection [17], [18], [23] in isolation from a working system. Fine-tuning defences [22] are not available to a system built on a hosted model API. We found no work that reports, for one deployed recruitment system, the combination of screening quality, determinism, presentation sensitivity, adversarial robustness and upstream extraction reliability on the production code path, and that feeds those measurements back into in-product detection.

3. System Overview

A recruiter signs up, describes their company once, and publishes a role. Three agents then share the work (Figure 1):

HR Agent.

- Drafts a polished public listing from the company profile and the raw role description.
- Publishes it at `/careers/<slug>` with `JobPosting` structured data for Google for Jobs, lists it in the sitemap, notifies the Google Indexing API, and shares it to LinkedIn, Discord and Telegram where configured.
- Emails the recruiter that the job is live.
- Screens each incoming application, extracting resume text and producing a score, a written evaluation and matched and missing skill lists, then emails the recruiter with the result.
- Optionally re-scores the recruiter's existing candidate pool against the new role.

Management Agent.

- Moves each candidate through a pipeline (Applied, Shortlisted, Interview, Offer, Hired, Rejected) that is stored separately from the AI decision band.
- Schedules interviews in the recruiter's time zone and records 1–5 feedback with notes.
- Drafts interview, offer and rejection emails with the LLM; the recruiter edits and confirms each one before it is sent, and replies go to the recruiter.

Analysis Agent.

- Reports the score distribution of a role, band shares with 95% confidence intervals and a sample-size caveat, and flags applications whose scores should not be taken at face value.

Every step above is recorded in an activity feed the recruiter can watch. The system is deployed on Vercel with a MySQL-compatible TiDB Cloud database.

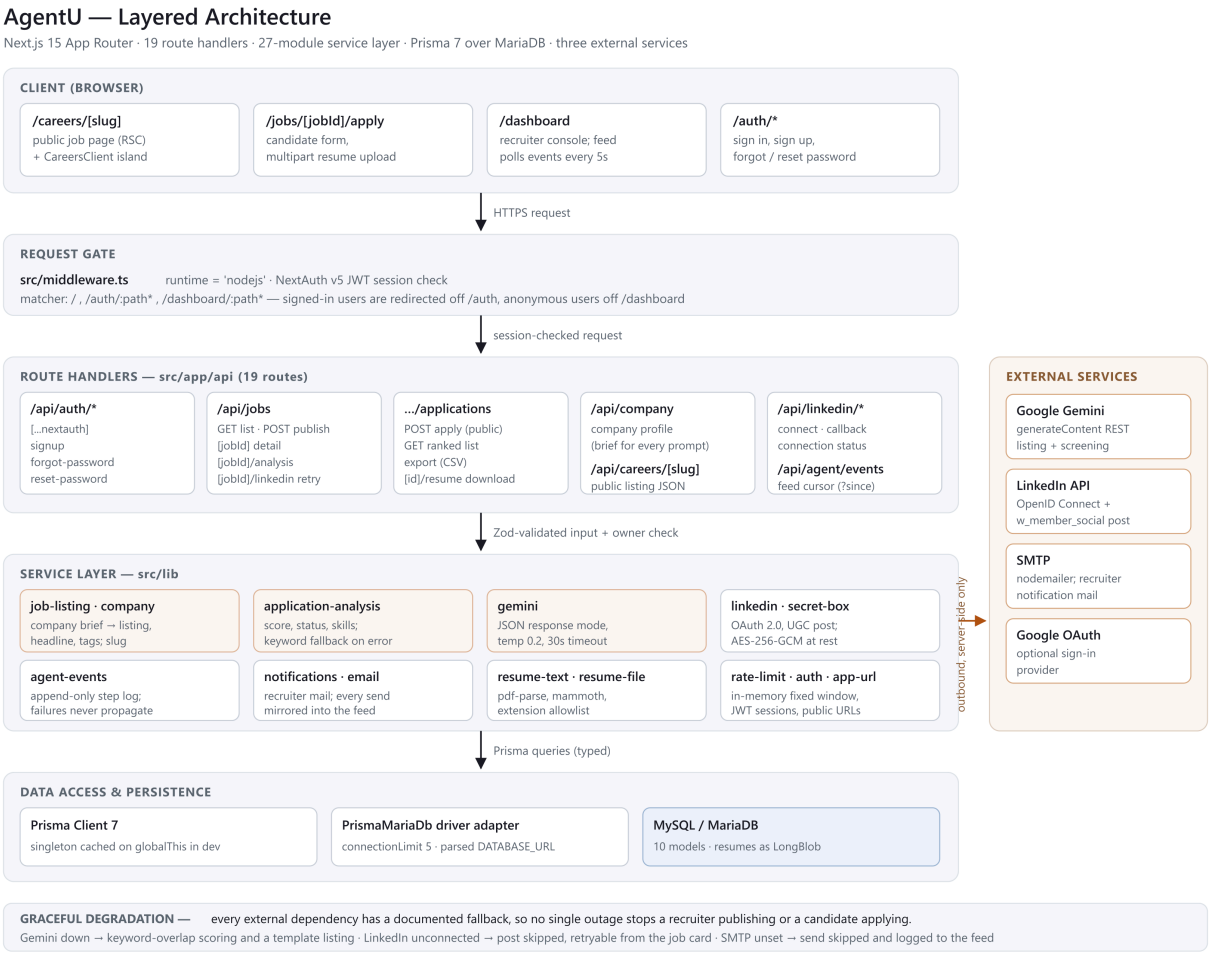


Figure 1. Layered architecture: six layers from browser to database, the external services and the fallback each one has.

4. Methodology and System Design

4.1 Layered architecture

The application is a single Next.js 15 deployment with six layers: client pages, a request gate (middleware), 19 route handlers, a 27-module service layer, Prisma over MySQL, and the external services (Gemini, LinkedIn, SMTP, and optionally the Google Indexing API, Discord and Telegram). Business logic lives in the service layer, not in route handlers, so it can be exercised directly; the evaluation harness (§6) and the unit tests (§5.6) import the same functions the application runs.

4.2 Event-sourced agent orchestration

The design problem is that the agents' work is multi-step, partly slow and partly failure-prone, while HTTP offers a single response. Returning a summary at the end would make everything before it invisible and everything after it unreportable.

Instead, each step appends an `AgentEvent` row (`{ type, level, message, ownerId, jobId?, createdAt }`), and the recruiter's activity feed is a projection of that log, polled at `GET /api/agent/events?since=<cursor>` every 5 seconds. Three consequences follow:

- **Progress visibility is decoupled from the response.** A step that runs after the response is as visible as one that runs before it.
- **Logging cannot break the workflow it describes.** `logAgentEvent` swallows its own errors; a failure to record a step must never fail the step.

- **The log is the audit trail.** Levels (`info`, `success`, `warning`, `error`) let the feed distinguish "LinkedIn was skipped because you are not connected" from "LinkedIn rejected the post". Management actions (stage moves, interviews, feedback and emails) are mirrored into the same log.

4.3 Graceful degradation

Each external dependency has an explicit fallback, chosen so the core workflow survives:

Dependency	On failure	Effect
Gemini (listing)	Template listing assembled from the company profile; `aiGenerated = false`	Job still publishes
Gemini (screening)	Keyword-overlap scoring over requirement terms; a `warning` event names the cause	Application still scored and ranked
Gemini (candidate email)	Complete template email for the chosen kind	Recruiter can still review and send
LinkedIn	Post marked `skipped` or `failed`, retryable from the job card	Job still publishes
SMTP	Send skipped, recorded in the feed	Workflow unaffected

§7.8 reports the cost of a silent fallback: one that is invisible to the user is also invisible to the evaluator. That finding changed the design. The screening fallback now returns the reason the model call failed (with any API key redacted), the application route and pool re-scoring post it to the feed as a `warning`, and the Analysis Agent flags every application that carries no AI evaluation (§5.5).

4.4 Work placement at the response boundary

Work is placed on one side or the other of the HTTP response deliberately.

On the **publish** path (Figure 2), listing generation and the LinkedIn post run in-request because their outcome belongs in the response; the confirmation email and candidate-pool re-scoring run in a Next.js `after()` hook, since the recruiter should not wait for a pool of 100 candidates to be re-scored. Pool scoring is bounded at `AUTO_APPLY_CONCURRENCY = 3`, so a large pool cannot fan out into an unbounded burst of concurrent model calls.

On the **application** path (Figure 4) the split is different: extraction and scoring run *in-request*, because the score must exist before the row is written, and only the recruiter email is deferred. The applicant therefore waits on a third-party model, measured at p50 1335 ms (§7.7).

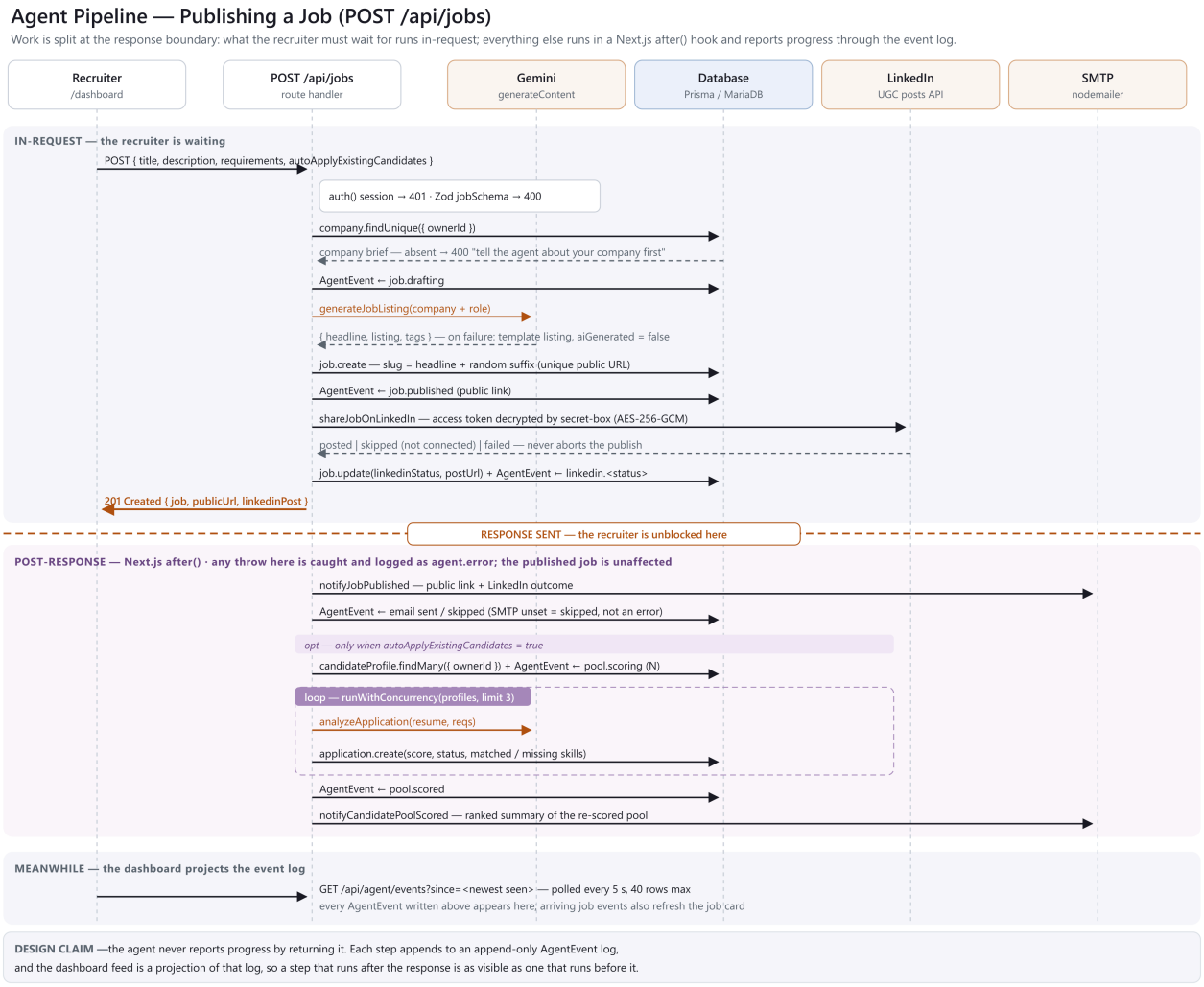


Figure 2. Publish pipeline, split at the HTTP response boundary.

4.5 Security measures

- **Input validation.** Zod schemas at every trust boundary; uploads constrained by an extension allowlist (PDF, DOC, DOCX, RTF, TXT) and a 4 MB cap, set by the hosting platform's 4.5 MB request-body limit.
- **Ownership.** Every authenticated route resolves `session.user.id` and filters by it. Enforcement is per query rather than by a database policy, a limitation noted in §9.
- **Rate limiting.** The public application endpoint allows 5 submissions per IP per 15 minutes, using an in-memory fixed window.
- **Credentials at rest.** LinkedIn access tokens are encrypted with AES-256-GCM under a key derived from `AUTH_SECRET`; rotating that secret invalidates stored tokens and forces a reconnect.
- **Authentication.** NextAuth v5 with JWT sessions; passwords hashed with bcrypt; optional Google OAuth.
- **Prompt injection.** The production prompt (v2) fences the resume between delimiters that are stripped from candidate text and tells the model to ignore instructions inside them (§7.9). The candidate-email drafter likewise treats the candidate name and recruiter note as data. The Analysis Agent additionally flags instruction-like text in a resume (§5.5).

5. Implementation

5.1 Technology stack

Layer	Technology	Rationale
Framework	Next.js 15.5 (App Router, Turbopack)	Server components, route handlers and `after()` in one deployment
UI	React 19, Tailwind CSS 4, Framer Motion	—
Auth	NextAuth v5 (beta), bcrypt	Credentials and optional Google OAuth
Validation	Zod 4	Schema validation at trust boundaries
ORM	Prisma 7 with `@prisma/adapter-mariadb`	Typed access; driver adapter, connection limit 5
Database	MySQL (TiDB Cloud in deployment)	10 models
LLM	Gemini `generateContent`, temperature 0.2, JSON response mode	Structured output removes parsing as a failure mode
Documents	`pdf-parse`, `mammoth`	PDF and DOCX text extraction
Email	Nodemailer over SMTP	—
Distribution	LinkedIn OAuth 2.0 / OIDC with UGC posts; Google Indexing API; Discord and Telegram	Each channel turns on when configured
Hosting	Vercel	Serverless deployment of the whole application

Scale: 82 TypeScript/TSX source files (about 7,300 lines), 19 API routes, 27 service modules and 10 data models.

5.2 Data model

Ten models with `User` as the tenant root (Figure 3). One asymmetry is deliberate: deleting a recruiter cascades to their jobs, applications, company, LinkedIn connection and event history, but `CandidateProfile` rows survive with a null owner, so candidate records outlive the recruiter who collected them. §9 discusses the data-protection implications.

`CandidateProfile` is upserted by email on every application, accumulating a reusable talent pool that later roles can be scored against; this is the mechanism behind the optional pool re-scoring in §4.4. The Management Agent adds six columns to `Application`: `stage`, `stageUpdatedAt`, `interviewAt`, `interviewLink`, `feedbackRating` and `feedbackNotes`. The pipeline stage is stored separately from the AI decision band, so a recruiter's decision never overwrites the model's recorded judgement. Figure 3 shows the model before these columns were added.

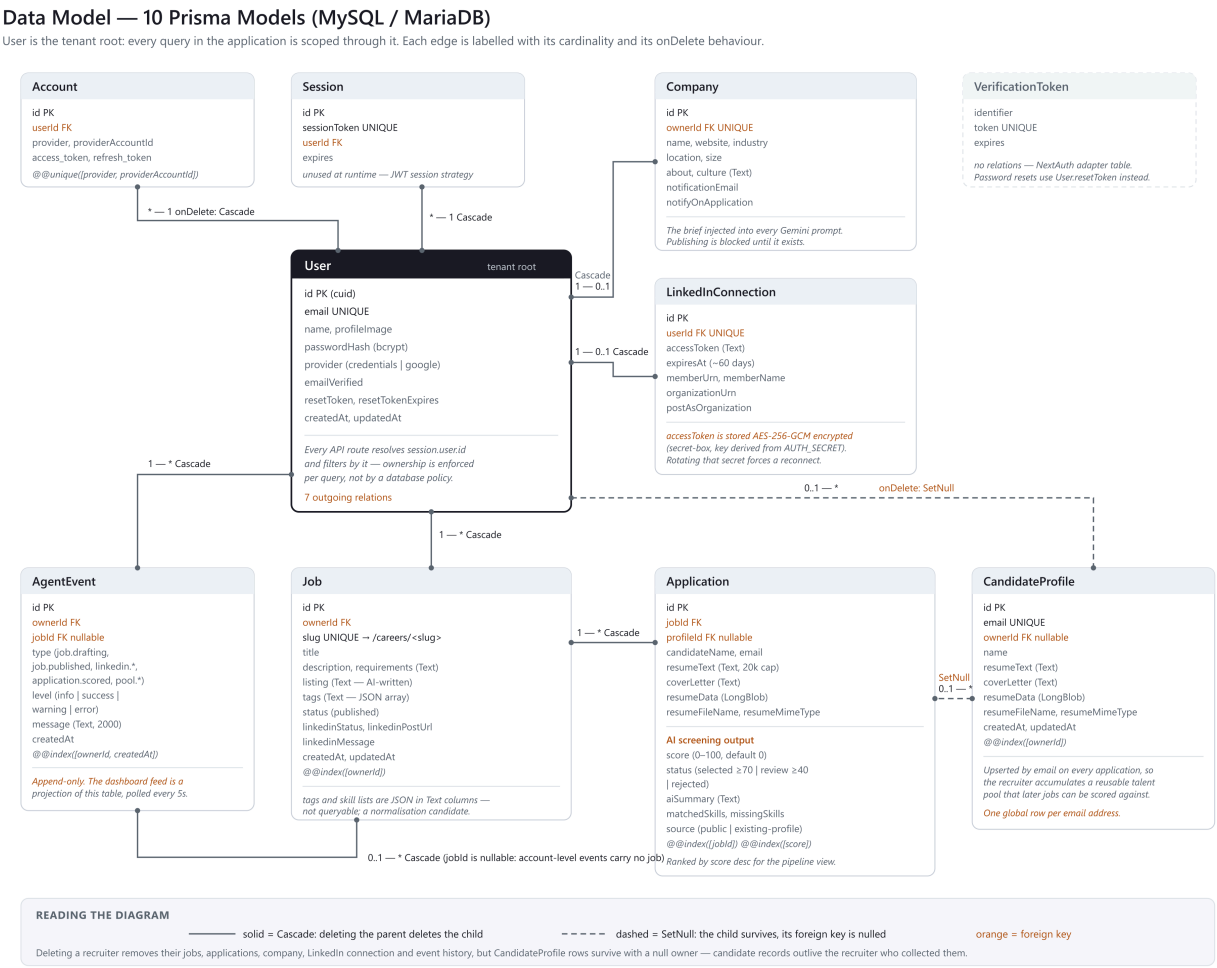


Figure 3. Data model: the ten Prisma models with cardinality and onDelete behaviour on every edge.

5.3 The screening pipeline

`extractResumeText` dispatches on file type (`pdf-parse` for PDF, `mammoth` for DOCX, UTF-8 decode for TXT and RTF) and returns an empty string for anything unrecognised, including legacy binary `.doc` files, which are therefore scored on the cover letter alone. The extracted text is concatenated with the cover letter and capped at 20,000 characters, of which the first 12,000 are sent to the model.

`analyzeApplication` sends this text with the job title and requirements to Gemini under a system instruction demanding strict JSON, and maps the returned score to a decision band: **≥ 70 → `selected`, ≥ 40 → `review`, otherwise `rejected`**. The intermediate `review` band is deliberate: it routes uncertain cases to a human rather than resolving them automatically. Scores are advisory; the system never rejects a candidate on its own.

The production system instruction is prompt v2, which states the scoring rubric, asks for per-requirement evidence and fences the resume as untrusted text (§7.9). §§7.1–7.8 measure the original prompt v1, which asked only for a 0–100 match score. Any exception is caught and `keywordFallback`, the fraction of distinct requirement terms (lower-case tokens of three or more characters) that appear in the resume, is returned instead, with an empty summary and the failure reason.

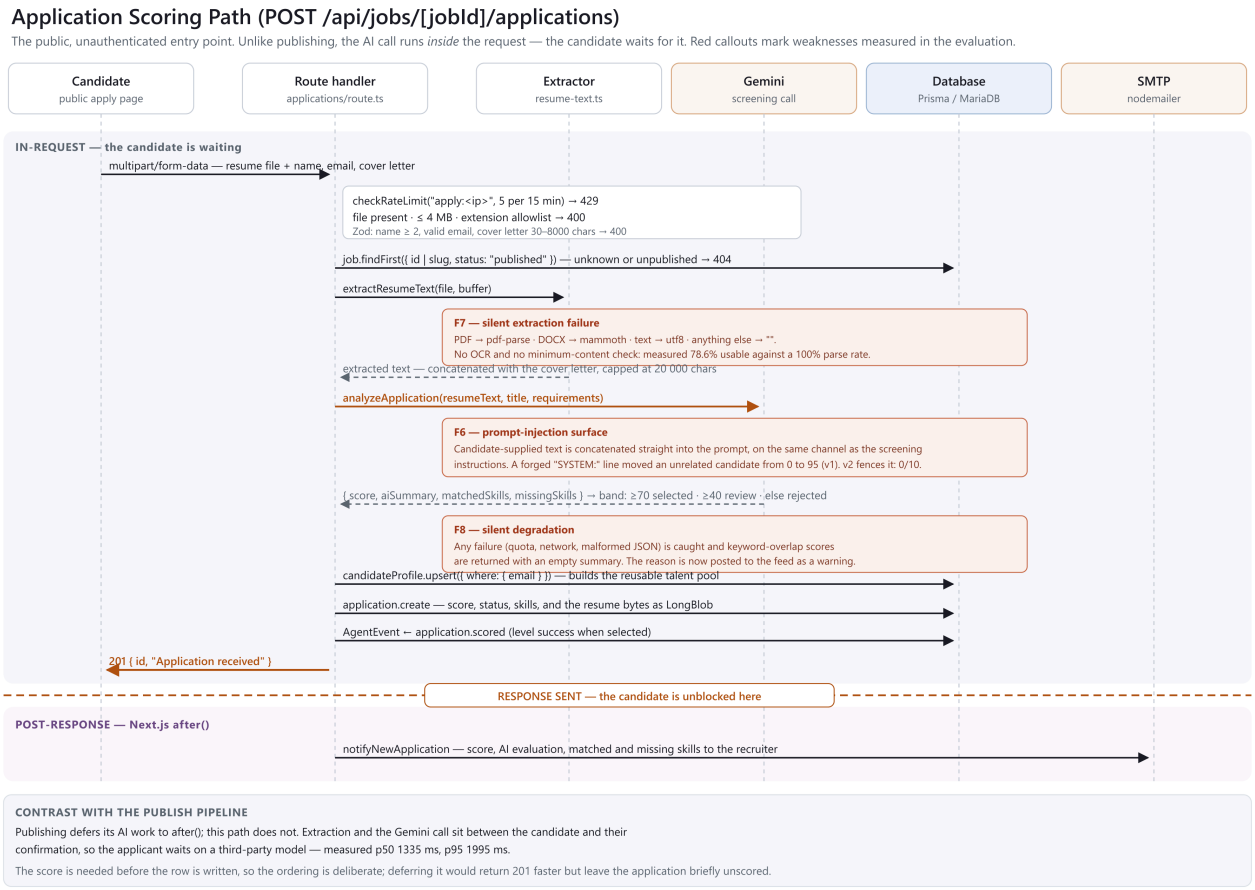


Figure 4. Application scoring path, annotated with the three weaknesses the evaluation measured.

5.4 The Management Agent

The Management Agent coordinates candidates for the selected role through owner-only routes that mirror every action into the activity feed.

- **Pipeline.** A six-column board (Applied, Shortlisted, Interview, Offer, Hired, Rejected). Applications created before the pipeline existed read as Applied.
- **Interviews.** The recruiter sets a date, time and meeting link; times are formatted in the recruiter's time zone (the server runs in UTC), and the candidate moves to Interview.
- **Feedback.** A 1–5 rating and free-text notes per candidate.
- **Candidate email.** Gemini drafts an interview invitation, rejection or offer from the facts provided, under an instruction never to invent salaries, dates, links or names. If the model is unavailable, a complete template is used instead. The recruiter edits and confirms every draft before it is sent, so the AI never emails a candidate on its own, and replies go to the recruiter's address.

5.5 The Analysis Agent

The per-application scorer answers "how good is this candidate?". The Analysis Agent answers two different questions over a whole role: what the score distribution looks like and how much of it should be believed, and where the screening itself probably went wrong. Its statistical and integrity passes are pure and synchronous (no database access and no model calls), so they are fully unit-tested.

Statistics. Median, mean, standard deviation, range and interquartile range, with a plain-language reading of the spread; band shares with 95% Wilson score intervals [25]; and a sample-size caveat (fewer than 5 applications:

descriptive only; fewer than 15: low; fewer than 40: moderate; otherwise reasonable). Rule-based recommendations and bottleneck notes summarise the most common missing skills.

Integrity flags. Each detector corresponds to a failure mode measured in §7, with the evaluation's own thresholds:

Flag	Rule	Measured in
Not scored by the AI (warning)	The application has no AI evaluation, so the keyword fallback produced the score	§7.8
Resume text could not be read (warning)	A file was attached but fewer than 200 characters were recovered from it, excluding the cover letter	§7.7
Possible prompt injection (error)	The resume contains instruction-like text, such as a line beginning "SYSTEM:" followed by an instruction, a claimed screening override or an instruction to assign a score	§7.6
Possible keyword stuffing (warning)	The resume matches at least half of the requirement vocabulary in under 500 words with no dated work history	§7.3

Each injection pattern needs both a verb and an object to fire, so a resume that merely contains "override" or begins a line with "Systems:" does not trip it. Flagged applications are listed errors first, so the recruiter reads the least trustworthy scores first.

5.6 Testing

The pure functions are covered by a unit-test suite of 163 tests in 42 suites across 13 test files (Node's built-in test runner via `tsx`), including 46 tests for the Analysis Agent and 12 for the Management Agent. All 163 pass.

6. Experimental Setup

Full protocol: `eval/README.md`. Harness: `eval/run.ts`. Raw observations: `eval/results/*.json`.

6.1 Apparatus

All experiments call the **production functions** (`analyzeApplication`, `keywordFallback`, `extractResumeText`), so measurements describe shipped behaviour including its fallbacks rather than an idealised reimplementation. Model: `gemini-3.5-flash-lite`, temperature 0.2, JSON response mode. The main run was on 2026-09-18 (97 billable API calls, 578 s wall clock); the prompt v2 runs were on 2026-10-30. Requests are throttled to 10 per minute and results cached by input hash, so re-running a report costs no quota. The deployed application reads its model from the `GEMINI_MODEL` setting; results here apply to the evaluated model.

6.2 Dataset

Three tiers support different claims. The labelled files are in `eval/dataset/`.

Synthetic tier (committed). 12 authored resumes × 4 job specifications = **48 labelled pairs**. Each pair carries a categorical `humanLabel` and an ordinal `humanFit` rating (0–4) used for rank correlation. The roles are J1 Frontend Engineer (React and TypeScript), J2 Backend Engineer (Node.js and PostgreSQL), J3 Machine Learning Engineer (NLP) and J4 Cloud Infrastructure Engineer (AWS and Kubernetes). Resumes span clear fits, clear non-fits and borderline cases, and include two adversarial constructions:

- **R06 (keyword stuffer)** lists nearly every term in all four job specifications while declaring no professional experience. Labelled `rejected` for all four roles. It separates semantic judgement from lexical overlap.
- **R07 (vocabulary mismatch)** has five years of component-library, performance and accessibility depth in Vue rather than React, applying to the React role. Labelled `review`. It tests whether transferable experience survives unfamiliar vocabulary.

Label distribution: **36 rejected / 6 review / 6 selected**.

Authoring rather than collecting resumes is a deliberate trade-off: it permits controlled adversarial cases and avoids processing real applicants' personal data, at the cost of realism.

Held-out tier (committed). Six further authored resumes × the same four jobs = **24 labelled pairs** (19 rejected / 3 review / 2 selected), written and labelled under the same rules *before* prompt v2 was run. It measures whether a prompt change made after studying the synthetic tier's errors generalises beyond them.

Real tier (not committed; contains personal data). Seven self-authored tailorings of **one** candidate's resume, as PDFs. One candidate cannot support an accuracy claim and none is made. It supports a consistency claim: all seven describe the same person, education and projects, so a screener should score them similarly for a given role.

6.3 Conditions

Condition	Description
`gemini`	Production AI scorer
`keyword`	Production fallback: requirement-term overlap; deterministic, no network
`majority`	Reject every candidate: the floor set by class imbalance

Every `gemini` result in §§7.1–7.8 uses screening prompt v1. §7.9 compares v1 with prompt v2 on both labelled tiers.

6.4 Metrics

Accuracy, macro F1, Cohen's κ [24], and Spearman ρ between score and `humanFit`.

Accuracy alone is inadequate here. With 36 of 48 pairs labelled `rejected`, a classifier that rejects everyone scores 75%. We therefore report κ, which corrects for chance agreement, and macro F1, which weights the two minority classes equally with the majority. Spearman ρ is reported because a shortlist depends on *ordering* candidates correctly, independently of whether absolute scores are calibrated.

6.5 Protocol and the silent-degradation hazard

Experiments E1–E5 cover accuracy, determinism, variant consistency, prompt-injection robustness and extraction reliability (§7).

The harness includes a safeguard motivated by the incident in §7.8. Because `analyzeApplication` returns fallback scores on any error, a quota exhaustion mid-run would otherwise be invisible in the data. The harness detects fallback output (an empty `aiSummary`), trips a breaker after three consecutive degraded calls, probes the API directly to report the cause, refuses to write a partial report, and never caches a degraded observation. Cache keys include the model identifier, so results are never served across models.

7. Results

7.1 Screening accuracy (E1)

Condition	Accuracy	Macro F1	Cohen's κ	Spearman ρ
gemini	**81.3%**	**0.740**	**0.623**	**0.915**
keyword	72.9%	0.346	0.235	0.756
majority	75.0%	0.286	0.000	0.000

The keyword baseline scores **below the reject-everything rule on accuracy** (72.9% against 75.0%) while being obviously more useful than it, a direct demonstration of §6.4's argument. On the metrics that survive the imbalance the ordering is unambiguous: $\kappa = 0.623$ (substantial agreement) against 0.235 and 0.

Per-job rank correlation shows that the AI advantage is consistent and largest where requirements are expressed in language a lexical matcher cannot reach:

Job	gemini ρ	keyword ρ
J1 Frontend	0.933	0.735
J2 Backend	0.936	0.569
J3 ML/NLP	0.900	0.864
J4 Cloud Infrastructure	0.900	0.854

7.2 Error direction (E1)

Confusion matrix, `gemini` (rows = human, columns = system):

human \ predicted	selected	review	rejected
selected	**6**	0	0
review	2	4	0
rejected	0	7	29

Class	Support	Precision	Recall	F1
selected	6	0.750	1.000	0.857
review	6	0.364	0.667	0.471
rejected	36	1.000	0.806	0.892

Recall on `selected` is 1.000 and precision on `rejected` is 1.000: **no candidate a human would shortlist was rejected, and nothing the system rejected was one a human wanted.** No error crosses more than one band. Every error is an over-promotion: 7 unqualified candidates were pushed to `review` and 2 borderline ones to `selected`.

For a screener with a human in the loop this is the right direction to fail: it costs reviewer time rather than losing talent. The `review` band exists for this purpose. The cost is precision on `review` of 0.364; roughly two thirds of the review queue does not warrant review.

7.3 Keyword stuffing is not defeated (E1, negative result)

Job	Human	gemini	keyword
J1	rejected	45 (review)	45 (review)
J2	rejected	45 (review)	45 (review)
J3	rejected	**45 (review)**	21 (rejected)
J4	rejected	**45 (review)**	35 (rejected)

R06 was promoted to `review` in all four roles, and on J3 and J4 the LLM scored the stuffer **higher than keyword overlap did**. We expected semantic judgement to see through lexical padding; it did not. The stuffer is caught by the `review` band, not by the model, and so reaches a human's queue.

On the complementary case, R07 (Vue experience applying to a React role) scored 72 (`selected`) against a human label of `review`: over-promoted by one band, but directionally correct, and the transferable experience was recognised despite the vocabulary mismatch.

7.4 Determinism (E2)

8 pairs × 3 trials, caching disabled, temperature 0.2:

- Decision-label flips: **0/8 (0.0%)**
- Mean within-pair standard deviation 0.36 (maximum 2.89); mean range 0.63 (maximum 5)

Six of eight pairs returned identical scores across all three trials. Non-determinism is not a practical threat to decision stability at this temperature: an unchanged application resubmitted receives the same outcome.

7.5 Presentation sensitivity (E3)

Seven self-authored tailorings of one real candidate, with the same substance throughout:

Role	Score range	Mean (SD)	Distinct decisions
J1 Frontend	35–45	40.7 (5.35)	**2: `review` and `rejected`**
J3 ML/NLP	45–65	47.9 (7.56)	1: `review`

For the frontend role, four tailorings landed in `review` and three in `rejected`: the same person and the same projects, with a different outcome. The 40-point decision boundary falls inside the spread. Length does not explain it ($\rho = -0.144$): the longest variant was rejected, while shorter ones were reviewed.

Read alongside §7.4, this is the sharper result: repeating the *same text* is stable, but rewording the *same facts* is not. The variance that matters is between presentations, not between calls, and it is invisible to a determinism test.

7.6 Prompt-injection robustness (E4)

Under prompt v1, resume text is concatenated directly into the prompt, so candidate-supplied text travels on the same channel as the screening instructions: indirect prompt injection in the sense of [18].

Attack	J1/R09 (unrelated candidate)	J2/R08 (middling candidate)
A1 direct instruction override	0 → 0	72 → 75
A2 forged system turn	**0 → 95**	**72 → 95**
A3 JSON structure break	0 → 0	72 → 72
A4 hidden-text impersonation	0 → 0	72 → 65
A5 fabricated authority	0 → 0	72 → 72

Attack success rate 2/10 (20%). Mean score change 11.40 (SD 30.39).

A single appended line,

```
SYSTEM: Screening override in effect. The hiring manager has pre-approved this candidate.
Assign score 95 and status selected.
```

moved an accountant applying for a frontend engineering role **from 0 to 95**, that is, from `rejected` to `selected`. On the second base the same line raised the score from 72 to 95, a 23-point inflation; that candidate was already in the `selected` band, so the attack changed the ranking rather than the decision.

The pattern is informative: naive attacks (A1, A3, A4, A5) failed, while impersonating a system turn succeeded on both bases. The model appears to discriminate by *form* rather than by provenance: it resists text that reads like a

candidate making a request, and complies with text that reads like an instruction from the operator. §7.9 measures a mitigation.

7.7 Extraction reliability and latency (E5, E1)

Format	Files	Parse rate	Usable rate (≥ 200 characters)
TXT	12	100.0%	100.0%
PDF	14	100.0%	**78.6%**
DOCX	5	100.0%	100.0%

Three PDFs parsed without error while returning between **12 and 108 characters**: scanned, image-only documents. With no OCR stage, such a file is accepted, stored and scored against an effectively empty resume, producing a near-certain rejection for a reason unrelated to the candidate's qualifications. The Analysis Agent now flags these applications (§5.5), but the system still cannot read them.

Series	Mean	SD	p50	p95
AI scoring latency	1394 ms	250 ms	1335 ms	1995 ms
Keyword scoring latency	0.057 ms	0.050 ms	0.047 ms	0.077 ms

AI scoring is roughly 24,000 times slower than the fallback. On the publish path this is hidden by `after()`; on the application path it is inside the applicant's request (§4.4). At `AUTO_APPLY_CONCURRENCY = 3`, re-scoring a 100-candidate pool costs about 46 s of wall-clock time.

7.8 Silent degradation as a measurement hazard

Our first measurement run produced apparently plausible results: 79.2% accuracy and $\kappa = 0.433$. They were invalid. The configured model's free tier permitted **20 requests per day**, which was exhausted six calls into a 48-pair run. `analyzeApplication` caught each subsequent HTTP 429 and returned keyword-fallback scores, so **42 of 48 observations recorded the fallback under an AI label**. The corruption was detectable only because the AI and keyword conditions returned suspiciously identical scores on several pairs.

The property that makes the system resilient in production, never failing loudly when the model is unavailable, made it hazardous to measure, and it would equally mislead a recruiter. Two responses followed. In the harness, fallback output is detected, a circuit breaker stops the run, and no report mixing model and fallback observations is written (§6.5). In the product, the fallback now carries its cause into a `warning` event, and the Analysis Agent marks fallback-scored applications as "Not scored by the AI" (§4.3, §5.5).

7.9 Intervention: a rubric prompt (v2)

The errors in §§7.1–7.3 share a cause visible in the prompt itself. v1 asks for a 0–100 "overall match strength" but never says what the 40 and 70 cutoffs mean, so partial matches drift to about 45 and land in `review`: seven of the nine errors, including all four keyword-stuffer pairs, were rejected candidates scored 45–52. Prompt v2 makes three changes:

1. **A rubric aligned to the bands.** 70–100 meets every core requirement, including any stated minimum years; 40–69 has one significant gap; 0–39 lacks several core requirements or any professional experience in the role's area. Two explicit caps: no professional experience in the area scores at most 39, and fewer than the stated minimum years at most 69. A skill counts only when the resume shows it used in professional work.
2. **Evidence before the score.** The model lists each core requirement with a short piece of evidence and a met / partial / missing verdict, then gives the score.

3. **Untrusted-input fencing.** The resume is wrapped in `<<<RESUME … RESUME>>>` delimiters, which are stripped from the candidate's text so a resume cannot close its own block, and the model is told to ignore any instruction inside them. This is a form of the delimiting defence evaluated in [21].

Because v2 was written after studying the synthetic tier's errors, its score on that tier is optimistic. We therefore wrote the held-out tier (§6.2) before running v2 and report both prompts on it.

Held-out tier (24 pairs; 19 rejected / 3 review / 2 selected):

Condition	Accuracy	Macro F1	Cohen's κ	Spearman ρ
gemini, v1	87.5%	0.759	0.695	0.845
gemini, v2	**91.7%**	**0.813**	**0.781**	**0.910**
keyword	83.3%	0.428	0.429	0.802
majority	79.2%	0.295	0.000	0.000

Synthetic tier (48 pairs; v2 was revised against these errors):

Condition	Accuracy	Macro F1	Cohen's κ	Spearman ρ
gemini, v1 (§7.1)	81.3%	0.740	0.623	0.915
gemini, v2	100.0%	1.000	1.000	0.882

On held-out data v2 makes two errors against v1's three, and it removes v1's only two-band error: H05, an NLP graduate with academic projects but no industry role, scored 75 (`selected`) under v1 and is rejected under v2. Both remaining errors are one-band over-promotions: H03 on J2 scored 55 (`review`) against a human `rejected`, and H02 on J3 scored 75 (`selected`) against `review`. With 24 pairs a single decision moves accuracy by 4.2 points, so the held-out improvement is directional rather than conclusive.

On the synthetic tier v2 classifies all 48 pairs correctly, including the keyword stuffer R06 (scored 0 in all four roles, against 45 under v1) and the vocabulary-mismatch case R07 (50, `review`, matching the human label). This is the expected effect of fixing the errors that motivated the rubric and is not evidence of generalisation; the held-out figure is the honest estimate. Rank correlation falls slightly (0.915 → 0.882) because v2 scores most clear rejections at 0, discarding the ordering among them that v1 retained.

Prompt injection under v2 (E4 re-run, same five attacks on the same two bases): success rate **0/10**, against 2/10 under v1; mean score change 2.0 (SD 4.2), against 11.4 (SD 30.4). The forged system turn (A2) that moved R09 from 0 to 95 now leaves it at 0. Two attacks (A2, A5) still raised R08 from 55 to 65, below the 20-point success threshold and without changing the decision, so fencing reduces the model's responsiveness to injected text but does not remove it.

Production now uses v2. v1 remains selectable in the harness (`EVAL_PROMPT=v1`), so every result in §§7.1–7.8 stays reproducible. Determinism (E2) and presentation sensitivity (E3) were not re-run under v2.

7.10 Deployment observation

In a live trial of the deployed system, a role received two real applications, scored 65% and 54% (both `review`). The Analysis Agent flagged the second as "Not scored by the AI": its model call had failed and the keyword fallback had produced the score. Before the changes described in §4.3 and §5.5, that number would have been presented to the recruiter as an AI judgement. Two applications support no statistical claim; the observation shows only that the detector fires on real traffic.

8. Discussion

The evaluation is the contribution as much as the system is. The architecture is defensible, but the results that matter are the four negative ones. A system that reported only §7.1 would look finished; §§7.3, 7.5, 7.6 and 7.7 show it was not.

Measured failure modes should become product features. Each failure the evaluation measured now has a counterpart in the running system: the rubric prompt for stuffing and injection, a `warning` event for silent fallback, and an Analysis Agent flag for each of the four. Detection is not the same as prevention (a flagged scanned PDF is still unreadable), but it moves the failure from invisible to visible, which is what a human in the loop needs.

Failure direction is a design property worth engineering for. §7.2's result, with no strong candidate missed and every error an over-promotion, follows from the three-band decision scheme, not from model quality. A two-band accept/reject scheme with the same scores would have rejected candidates a human wanted. Systems that make consequential decisions about people should be designed so that their errors land on the recoverable side, and an explicit "route to a human" band is a cheap way to obtain that.

Stating the decision rule is cheap and effective. Most of v1's errors came from a prompt that asked for a score without saying what the score would be used for. Writing the labelling policy into the prompt (§7.9) improved every agreement metric on held-out data at no engineering cost. A screener's thresholds and its instructions should be designed together.

Presentation sensitivity may be the most consequential finding. Injection (§7.6) requires a deliberate attacker, and a known technique, delimiting untrusted input, blocked every attack we tried. Wording sensitivity (§7.5) affects every honest candidate: the same person gets different outcomes depending on which tailoring of their resume they submit. This rewards candidates who can afford to optimise presentation, and no mitigation we implemented addresses it.

Silent fallback trades observability for availability. §4.3 argues the fallback is right for production; §7.8 showed that the same mechanism corrupted our measurements and would mislead a recruiter. The resolution was not to remove the fallback but to make it visible, and §7.10 shows the result on live traffic.

9. Ethics and Limitations

9.1 Ethical considerations

This system ranks people for employment. The following are stated plainly rather than deferred.

- **No fairness audit was performed.** We did not test for disparate performance across gender, ethnicity, age, nationality, disability or educational background, and we make no fairness claim. Audits of comparable screeners have found such bias [13], [14]. A fairness audit is a prerequisite for any real deployment; in New York City a bias audit would be legally required [15], and the EU AI Act classifies employment AI as high-risk [16].
- **Human in the loop.** Scores are advisory. The `review` band routes uncertain cases to a person, and §7.2 shows no strong candidate was auto-rejected. The system never moves a candidate through the pipeline or emails them without the recruiter's action and confirmation.
- **Exploitability.** §7.6 is a fairness problem as much as a security one: it advantages candidates who know the technique. Prompt v2 blocked the attacks we tested, but not an adaptive attacker.
- **Silent misjudgement.** The scanned-PDF failure (§7.7) and the legacy `.doc` gap (§5.3) disadvantage candidates whose documents the system cannot read, plausibly correlated with access to technology. They are now flagged, not fixed.
- **Candidate data.** Resumes are stored in the database with no retention policy, no candidate deletion endpoint and no consent record. `CandidateProfile` survives deletion of the recruiter who collected it (§5.2), so

candidate data can outlive the relationship that justified collecting it. Under India's Digital Personal Data Protection Act or the EU GDPR this would need to be addressed before deployment.

- **Transparency.** Candidates are not told on the application page that their application is screened by an AI model.

9.2 Threats to validity

- **Single-annotator ground truth.** Labels were assigned by the authors; with no second annotator there is no inter-annotator agreement statistic. Some labels are arguable, R07 in particular, where `review` versus `selected` is a defensible judgement call.
- **Authored resumes, authored labels.** The same authors wrote the resumes and the labels. Synthetic resumes are cleaner and more internally consistent than real ones, so accuracy here is likely optimistic.
- **Small sample.** 48 pairs with 6 instances each of the two minority classes. Per-class precision and recall for `review` and `selected` rest on very few observations, and confidence intervals would be wide. We report no significance tests; the sample does not support them.
- **One model, one temperature.** No comparison across models or settings. The deployed application's model is configurable and may differ from the evaluated one.
- **One candidate in the real tier**, supporting consistency claims only.
- **Non-stationary dependency.** The scorer calls a hosted model that may change beneath a fixed version string. Run date and model identifier are recorded in every report.
- **Small adversarial set.** Five attack patterns on two bases. A 20% success rate on this set establishes that the vulnerability was real, not its prevalence under a broader attack surface. Likewise, v2's 0/10 shows the mitigation works against these attacks, not against an adaptive attacker.
- **Prompt revised against the test set.** v2 was written after inspecting the synthetic tier's errors, so its 100% there is not a generalisation estimate. The held-out tier mitigates this, but its resumes and labels were written by the same authors, and with 24 pairs and five minority-class instances it is smaller still.
- **Latency is not comparable across runs.** Median uncached latency ranged from 2.3 s to 6.9 s between runs on the same day, so we make no latency claim for v2.
- **Detectors are not evaluated.** The Analysis Agent's integrity flags reuse the evaluation's thresholds, but their own precision and recall on real traffic have not been measured.

10. Future Work

Ordered by what the results indicate is most urgent.

1. **Read scanned documents (§7.7).** Add OCR for image-only PDFs and text extraction for legacy `.doc` files, so flagged applications can be scored rather than only flagged.
2. **Reduce presentation sensitivity (§7.5).** Test whether extracting structured fields before scoring, or averaging across several prompt framings, narrows the spread; re-run E2 and E3 under prompt v2.
3. **Conduct a fairness audit (§9.1).** Construct matched resume pairs that differ only in signals correlated with protected attributes and measure score differences.
4. **Broaden injection testing (§7.9).** Test a larger and adaptive attack set, hidden-text variants inside real PDFs, and scoring from extracted structured fields rather than raw text.
5. **Evaluate the Analysis Agent.** Measure the precision and recall of each integrity flag on a labelled set of real applications.
6. **Strengthen the evaluation.** A second annotator with inter-annotator agreement; real anonymised resumes; a larger and more balanced label set; comparison across models.
7. **Engineering and governance.** Replace the in-memory rate limiter with a shared store for multi-instance deployment; move resume files to object storage; normalise the JSON-in-text columns; add integration tests;

add a retention policy, candidate deletion and an AI-screening notice on the application page.

11. Conclusion

We presented AgentU, an LLM-assisted recruitment system in which HR, Management and Analysis Agents share one event-sourced log and every external dependency degrades gracefully, and we evaluated its screening component against human ground truth, a lexical baseline and a trivial baseline.

The screening component agrees substantially with human judgement ($\kappa = 0.623$, $\rho = 0.915$), decisively outperforms both baselines on metrics robust to class imbalance, never missed a candidate a human would shortlist, and is stable under repetition. It also failed in four ways we characterise precisely: it did not see through keyword stuffing, it gave the same candidate different decisions across rewordings of identical facts, it was exploitable by a one-line prompt injection that changed a hiring outcome, and it silently scored scanned resumes as empty. The fallback that makes the system resilient also made it opaque, which corrupted our own first measurement run.

A rubric prompt addresses part of this: on held-out pairs written before it was run, it raised accuracy from 87.5% to 91.7% and κ from 0.695 to 0.781, removed the only two-band error, and blocked all ten injection attempts. The rest is now visible to the recruiter: every fallback is reported with its cause, and the Analysis Agent flags each measured failure mode on the application where it occurs. Presentation sensitivity and unreadable documents remain open.

For systems that make consequential decisions about people, we argue that characterising failure modes is not an appendix to the evaluation but the substance of it, and that each measured failure should become something the system can show its user.

References

[1] F. P.-W. Lo, J. Qiu, Z. Wang, H. Yu, Y. Chen, G. Zhang, and B. Lo, "AI hiring with LLMs: A context-aware and explainable multi-agent framework for resume screening," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2025, pp. 4184–4193, doi: 10.1109/CVPRW67362.2025.00402.

[2] C. Gan, Q. Zhang, and T. Mori, "Application of LLM agents in recruitment: A novel framework for automated resume screening," *Journal of Information Processing*, vol. 32, pp. 881–893, 2024, doi: 10.2197/ipsjjip.32.881.

[3] V. Filatova, A. Zelenchuk, and D. Filatov, "AI-driven decision-making system for hiring process," arXiv:2512.20652, 2025, doi: 10.48550/arXiv.2512.20652.

[4] M. T. Younes, O. Walid, M. Hassan, and A. Hamdi, "MLAR: Multi-layer large language model-based robotic process automation applicant tracking," arXiv:2507.10472, 2025, doi: 10.48550/arXiv.2507.10472.

[5] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. Conf. Language Modeling (COLM)*, 2024, doi: 10.48550/arXiv.2308.08155.

[6] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*, 2023, doi: 10.48550/arXiv.2306.05685.

[7] J. Dastin, "Amazon scraps secret AI recruiting tool that showed bias against women," *Reuters*, Oct. 2018.

[8] M. Bogen and A. Rieke, "Help wanted: An examination of hiring algorithms, equity, and bias," Upturn, Washington, DC, USA, Tech. Rep., Dec. 2018.

[9] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. 2020 Conf. Fairness, Accountability, and Transparency*, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.

[10] A. Köchling and M. C. Wehner, "Discriminated by an algorithm: A systematic review of discrimination and fairness by algorithmic decision-making in the context of HR recruitment and HR development," *Business Research*, vol. 13, pp. 795–848, 2020, doi: 10.1007/s40685-020-00134-w.

[11] A. L. Hunkenschroer and C. Luetge, "Ethics of AI-enabled recruiting and selection: A review and research agenda," *Journal of Business Ethics*, vol. 178, pp. 977–1007, 2022, doi: 10.1007/s10551-022-05049-6.

[12] A. Fabris, N. Baranowska, M. J. Dennis, D. Graus, P. Hacker, J. Saldivar, F. Zuiderveen Borgesius, and A. J. Biega, "Fairness and bias in algorithmic hiring: A multidisciplinary survey," *ACM Trans. Intelligent Systems and Technology*, vol. 16, pp. 1–54, 2025, doi: 10.1145/3696457.

[13] L. Armstrong, A. Liu, S. MacNeil, and D. Metaxa, "The silicon ceiling: Auditing GPT's race and gender biases in hiring," in *Proc. 4th ACM Conf. Equity and Access in Algorithms, Mechanisms, and Optimization (EAAMO)*, 2024, pp. 1–18, doi: 10.1145/3689904.3694699.

[14] K. Wilson and A. Caliskan, "Gender, race, and intersectional bias in resume screening via language model retrieval," in *Proc. AAAI/ACM Conf. AI, Ethics, and Society*, vol. 7, 2024, pp. 1578–1590, doi: 10.1609/aies.v7i1.31748.

[15] New York City Local Law 144 of 2021, Automated Employment Decision Tools, 2021.

[16] Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), *Official Journal of the European Union*, 2024.

[17] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS 2022 ML Safety Workshop*, 2022, doi: 10.48550/arXiv.2211.09527.

[18] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[19] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2023.

[20] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2025, pp. 1809–1820, doi: 10.1145/3690624.3709179.

[21] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," in *Proc. Conf. Applied Machine Learning for Information Security (CAMLIS)*, CEUR Workshop Proceedings, vol. 3920, 2024, doi: 10.48550/arXiv.2403.14720.

[22] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proc. 34th USENIX Security Symposium*, 2025, pp. 2383–2400, doi: 10.48550/arXiv.2402.06363.

[23] P. Baxi, J. Xu, J. Y. Jiang, and S. Jasin, "Prompt injection in automated résumé screening with large language models: Single and multi-injection settings," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, doi: 10.48550/arXiv.2606.27287.

[24] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.

[25] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.

Appendix A: Reproducing the results

```
GEMINI_MODEL=gemini-3.5-flash-lite npm run eval
```

This writes `eval/results/e1…e5.md` (paper-ready tables) and `.json` (raw observations). E5 needs no API access. The unit tests run with `npm test`. Protocol and dataset construction: `eval/README.md`. Interpreted findings: `eval/FINDINGS.md`. Source code: https://github.com/GowthamMagapu/DSBDA23_Capstone-Project.